

Informatika 3

Prednáška 4: OOP v jazyku C++

Michal Mrena



ŽILinskÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Programovacie paradigmy



Programovacie paradigmy

- Existencia objektovo orientovaného (hovorovo objektového) programovania implikuje existenciu iných prívlastkov
- Existuje mnoho rôznych **paradigiem**, predstavíme si vybrané:
- **Imperatívne programovanie**
 - Procedurálne programovanie
 - Objektovo orientované programovanie
- **Deklaratívne programovanie**
 - Funkcionálne programovanie
- Programovaním **riešime problémy**, paradigma hovorí o tom, **akým spôsobom** to robíme

Imperatívne programovanie

- Programovanie ako ho poznáte
- Postupnosťou **príkazov** (angl. statements) meníme **stav** programu (hodnoty premenných)
- Výsledok (riešenie problému) na konci prečítame ako hodnotu premennej
- Teoretický základ môžeme vidieť v [Turingovom stroji](#)



Procedurálne programovanie

- Program pozostáva zo sekvencie volaní procedúr/funkcii
- V procedúre meníme stav argumentov, lokálnych a globálnych premenných (aj keď sa vo všeobecnosti snažíme globálnemu stavu vyhýbať)
- Dáta a algoritmy sú oddelené – sú definované samostatne

Objektovo orientované programovanie

- Program pozostáva zo spolupracujúcich objektov
- Stav objektov sa mení v ich metódach
- Dáta a algoritmy, ktoré nad nimi operujú, sú definované spolu
- OOP môžeme ďalej deliť podľa toho, či je ústredným pojmom **trieda** alebo **objekt – prototyp**
- Z jazyka Java (a teraz už aj C++) poznáme prístup založený na triedach

Funkcionálne programovanie

- Postupným aplikovaním funkcií na hodnoty získavame výsledok (riešenie problému)
- **Stav** je limitovaný iba na hodnoty argumentov funkcií
- Funkcie nemajú **žiadne vedľajšie efekty** – závisia iba od hodnôt argumentov
- Teoretický základ môžeme vidieť v Lambda kalkuluse



Imperatívny prístup

```
#include <iostream>
#include <vector>

int sum(const std::vector<int> &xs) {
    int result = 0;
    for (int i = 0; i < xs.size(); ++i) {
        result += xs[i];
    }
    return result;
}

int main() {
    std::cout << sum({1,2,3,4,5,6,7,8,9}) << "\n";
}
```


Funkcionálny prístup

```
sum :: [Int] -> Int
sum [] = 0
sum (x:xs) = x + sum xs

main :: IO ()
main = putStrLn . show . sum $ [1..9]
```

Uvedený príklad je ilustratívny, v praxi by bolo potrebné adresovať niekoľko nedostatkov *

Jazyky a paradigmy

- Programovací **jazyk** a programovacia **paradigma** sú *v princípe* nezávislé
- O programovacom jazyku však vieme povedať, či je vhodné programovať v ňom v danej paradigme
- Jazyk môže obsahovať výrazové prostriedky, ktoré uľahčujú programovanie v danej paradigme
- Väčšina viacúčelových programovacích je **multiparadigmových**
- Napríklad v jazyku C++ môžeme pohodlne programovať procedurálne alebo objektovo a s využitím funkcionálnych prvkov
- Programovať v ňom čisto funkcionálne je možné, ale **nepraktické**

Von Neumannova architektúra

- Bez ohľadu na paradigmu náš program väčšinou* nakoniec vyzerá takto:

```
sum(std::vector<int> const&):  
    mov     rsi, QWORD PTR [rdi]  
    mov     rcx, QWORD PTR [rdi+8]  
    sub     rcx, rsi  
    je      .L7  
    sar     rcx, 2  
    lea     rax, [rcx-1]  
    cmp     rax, 2  
    jbe     .L8  
    mov     rdx, rcx  
    mov     rax, rsi  
    pxor    xmm0, xmm0  
    shr     rdx, 2  
    sal     rdx, 4  
    add     rdx, rsi
```

```
.Lc1R8:  
    movq    $.Lc1Rq_info, (%rbp)  
    movq    %rbx, %r14  
    jmp     Main_$wsum_info  
.Lc1Rh:  
    xorl    %ebx, %ebx  
    jmp     *(%rbp)  
.Lc1Rl:  
    leaq    Main_$wsum_closure(%rip), %rbx  
    jmp     *-8(%r13)  
.Lc1Rq_info:  
    movq    %rbx, %rax  
    movq    8(%rbp), %rbx  
    addq    %rax, %rbx  
    addq    $16, %rbp
```

https://en.wikipedia.org/wiki/Lisp_machine *

Triedy v jazyku C++



Triedy v jazyku C++

- Triedu definujeme klúčovým slovom `class`, identifikátorom a blokom, ktorý obsahuje deklarácie a definície jej **členov** (angl. class members)
- Pre členov definujeme ich modifikátory prístupu
 - `public` – dostupné pre všetkých
 - `protected` – dostupné v triede a v jej potomkoch
 - `private` – dostupné len v triede

```
class Player {  
public:  
    // verejní členovia  
  
protected:  
    // chránení členovia  
  
private:  
    // súkromní členovia  
};
```

Modifikátory prístupu

- Namiesto kľúčového slova `class` môžeme použiť aj kľúčové slovo `struct`
- Rozdiel je v predvolenom modifikátore prístupu:
 - `class` – ak neuvedieme žiadny implicitne sa aplikuje `private`
 - `struct` – ak neuvedíme žiadny implicitne sa aplikuje `public`
- `struct` zvykneme používať na reprezentáciu jednoduchých dátových tried/záznamov ako napríklad `Point3D` uvedený nižšie

```
struct Point3D {  
    int m_x;  
    int m_y;  
    int m_z;  
};
```

Členovia triedy

- (statické) funkcie – metódy
- (statické) premenné – atribúty
- Vnorené typy
- Vnorené triedy

Vnorené typy

- Alias typu dostupný v definícii triedy a v definíciách metód triedy
- Pri prístupe zvonku je potrebné uviesť aj meno triedy – možné iba pri modifikátore prístupu **public**

```
class Player {  
public:  
    using ScoreType = long long;  
  
    ScoreType getScore() const;  
    void addScore(ScoreType value);  
    void removeScore(ScoreType value);  
  
private:  
    ScoreType m_score;  
};  
  
int main() {  
    Player *p = new Player();  
    Player::ScoreType score = 10;  
    p->addScore(score);  
}
```


Vnorené typy

- Zaujímavosť pri modifikátore prístupu `private`
- Typ môžeme použiť, ale nemôžeme ho pomenovať

```
class Player {  
private:  
    using ScoreType = long long;  
};  
  
int main() {  
    Player *p = new Player();  
    Player::ScoreType score = p->getScore();  
}
```

```
<source>: In function 'int main()':  
error: 'using Player::ScoreType = long long  
int' is private within this context  
15 |     Player::ScoreType score = 10;
```

```
class Player {  
private:  
    using ScoreType = long long;  
};  
  
int main() {  
    Player *p = new Player();  
    auto score = p->getScore();  
}
```

```
OK: Compiler returned: 0
```

Vnorené triedy

- Trieda definovaná v tele inej triedy
- Používame pri triedach, ktoré sú úzko prepojené (angl. tight coupling)
- Pri pomenovaní typu vnorenej triedy platia rovnaké pravidlá ako pri vnorenom type
 - `Graph::Node`

```
class Graph {  
public:  
    struct Node {  
        int m_id;  
        double m_distance;  
    };  
  
    // ...  
  
private:  
    std::vector<Node> m_modes;  
};
```

Nestatické členské premenné

- Nestatické členské premenné deklarujeme zadáním ich typu a identifikátora
- Na mieste deklarácie môžeme uviesť predvolenú hodnotu
 - Odporúčame nemiešať s inicializáciou v konštruktore
 - Ak premennú inicializujeme aj v konštruktore, predvolená hodnota sa ignoruje

```
class Player {  
public:  
    // ...  
  
private:  
    int m_points {0};  
    bool m_alive = false;  
    double m_posX;  
    double m_posY;  
};
```

```
struct Point3D {  
    int m_x {0};  
    int m_y {0};  
    int m_z {0};  
};
```

Statické členské premenné

- Statické členské premenné **deklarujeme** zadaním ich typu a identifikátora za kľúčovým slovom **static**
- Na rozdiel od nestatických, ktoré sú **definované** automaticky pri vzniku inštancie, musíme statické premenné **definovať** v jednom zo zdrojových súborov
- Pri *celočíselných konštantách* môžeme uviesť inicializáciu pri deklarácii

display.hpp

```
class Display {  
public:  
    // ...  
  
private:  
    static const int N_ROWS = 8;  
  
private:  
    DisplayRow *m_rows[N_ROWS];  
};
```

display.cpp

```
int Display::N_ROWS = 8;
```

Cyklická závislosť

- V niektorých situáciách máme triedy vo vzťahu obojsmernej asociácie
- V príklade pri definícii triedy `Directory` nepoznáme triedu `File`
- Zmena poradia definícii problém nevyrieši
- Riešením je použitie pred-deklarácie (angl. forward declaration)

```
class Directory {  
public:  
    // ...  
  
private:  
    std::vector<Directory*> m_directories;  
    std::vector<File*> m_files;  
};  
  
class File {  
public:  
    // ...  
  
private:  
    Directory *m_parent;  
}
```

Pred-deklarácia

- Pred-deklarácia hovorí, že trieda s daným názvom bude existovať
- Až do bodu definície triedy je typ neúplný (angl. incomplete type)
- Ukazovateľ môže ukazovať na neúplný typ (prečo?)
- Pred-deklarácie môžeme použiť na zníženie závislostí medzi hlavičkovými súbormi

```
class File;  
  
class Directory {  
public:  
    // ...  
  
private:  
    std::vector<Directory*> m_directories;  
    std::vector<File*> m_files;  
};  
  
class File {  
public:  
    // ...  
  
private:  
    Directory *m_parent;  
}
```

Nestatické členské funkcie (metódy)

- Nestatické členské funkcie **deklarujeme** zadáním ich návratového typu, identifikátorom a zoznamu parametrov
- **Definujeme** ich v zdrojových súboroch, pričom v názve uvádzame aj názov triedy (vlastníka)

display.hpp

```
class Display {  
public:  
    void turnPixelOn(int row, int col);  
};
```

display.cpp

```
void Display::turnPixelOn(  
    int row,  
    int col  
) {  
    this->boundCheckExcept(row);  
    m_rows[row]->turnPixelOn(col);  
}
```

Nemenné (nestatické) členské funkcie

- Členskú funkciu, ktorá nikdy nemení stav inštancie nazývame **nemenná** (angl. immutable)
- V jazyku C++ môžeme túto vlastnosť explicitne vyjadriť a **vynútiť** kvalifikátorom `const`
- V nemennej funkcii nemôžeme modifikovať stav inštancie

```
class Display {  
public:  
    int getRowCount() const;  
    void flipPixel(int row, int col);  
};
```

```
int Display::getRowCount() const {  
    return N_ROWS;  
}
```

```
int Display::getRowCount() const {  
    m_rows[0] = nullptr;  
    return N_ROWS;  
}
```


Nemenné (nestatické) členské funkcie

- V nemennej funkcii je možné volať iba nemenné funkcie danej triedy
- Na inštancii, ktorej typ je `const`-kvalifikovaný môžeme volať iba nemenné metódy

```
class Display {  
public:  
    int getRowCount() const;  
    void flipPixel(int row, int col);  
};
```

```
int Display::getRowCount() const {  
    this->flipPixel(0,0);  
    return N_ROWS;  
}
```

```
int main() {  
    const Display *d = new Display();  
    d->flipPixel(0,0);  
}
```

Statické členské funkcie (metódy)

- Deklarujeme ich rovnako, ako nestatické členské funkcie s pridaním kľúčového slova `static`
- Definujeme ich rovnako, ako nestatické členské funkcie, bez kľúčového slova `static`
- Pri ich volaní uvádzame názov triedy a operátor ::

```
class GameOfLife {  
public:  
    static GameOfLife *createEmpty();  
};
```

```
GameOfLife *GameOfLife::createEmpty() {  
    return new GameOfLife();  
}
```

```
int main() {  
    GameOfLife *gol =  
        GameOfLife::createEmpty();  
}
```

Pamäťová reprezentácia tried

Triedy a inštancie v pamäti



Zarovnanie

- Pamäťová reprezentácia inštancií je ovplyvnená *zarovnaním*
- Každý typ má vlastnosť, ktorú nazývame zarovnanie – prirodzená mocnina čísla dva
- Zarovnanie hovorí koľko bajtov musí byť medzi dvomi po sebe idúcimi adresami na ktorých sa nachádza objekt daného typu
- Ak má typ zarovnanie, napríklad, 4, všetky objekty (premenné) daného typu budú na adrese, ktorá je násobkom 4

Zarovnanie

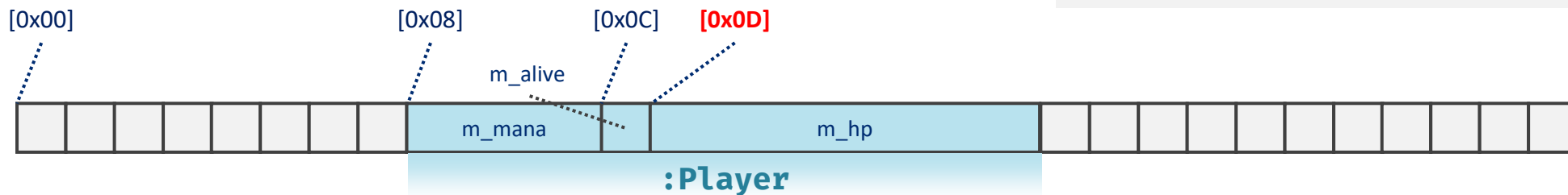
- Dôvod existencie potreby zarovnania nájdeme až na hardvérovej úrovni – súvisí so spôsobom akým CPU číta údaje z pamäte
- CPU číta z pamäte po slovách (napríklad 4 alebo 8 bajtov) – čítanie z nezarovnanej adresy by vyžadovalo dva prístupy do pamäte alebo, dokonca, nemusí byť niektorými architektúrami podporované
- Ďalšou úrovňou je cache pamäť a tzv. [cache lines](#) – pri čítaní slova sa z pamäte RAM načíta celý blok (typicky 64 bajtov) do cache pamäte L1
- Správne zarovnanie premenných zabezpečí kompilátor – ak sú na zásobníku alebo implementácia funkcií alokujúcich pamäť (operátor [new](#), [malloc](#), [calloc](#), ...)

Pamäťová reprezentácia inštancie

- Členské premenné sú uložené za sebou v poradí ich deklarácie
- Adresa inštancie je totožná s adresou prvej členskej premennej
- Môže byť premenná typu `double` na adrese `0x0D`?

```
class Player {  
public:  
    // ...  
  
private:  
    int m_mana;  
    bool m_alive;  
    double m_hp;  
};
```

```
int main() {  
    Player *p = new Player(); // p == 0x08  
}
```



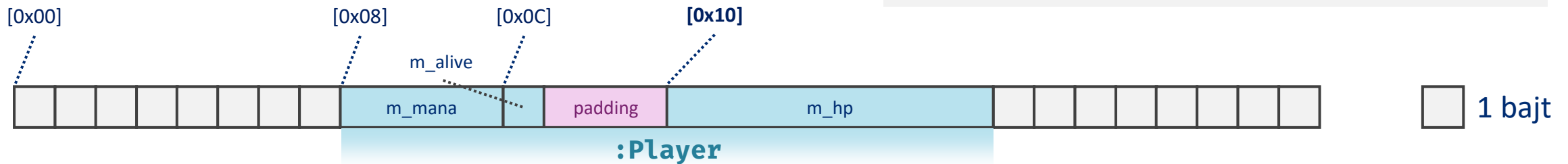
1 bajt

Pamäťová reprezentácia inštancie

- Zarovnanie typu `double` je 8
- Môže byť na adrese `0x08` alebo `0x10`, nie `0x0D`
- Riešenie: za premennú `m_alive` sa pridajú 3 bajty – **padding**

```
class Player {  
public:  
    // ...  
  
private:  
    int m_mana;  
    bool m_alive;  
    double m_hp;  
};
```

```
int main() {  
    Player *p = new Player(); // p == 0x08  
}
```

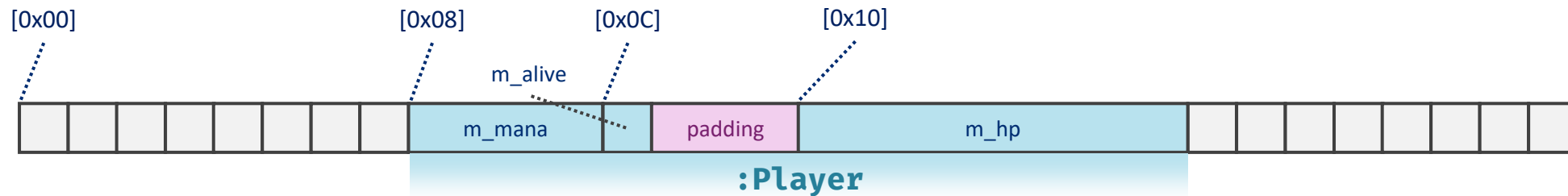


Pamäťová reprezentácia inštancie

- Zarovnanie typu získame operátorom `alignof`
- Aká je veľkosť a zarovnanie typu `Player`?

```
int main() {  
    std::cout << alignof(Player) << "\n"; // ?  
    std::cout << sizeof(Player) << "\n";  // ?  
}
```

```
class Player {  
public:  
    // ...  
  
private:  
    int m_mana;  
    bool m_alive;  
    double m_hp;  
};
```



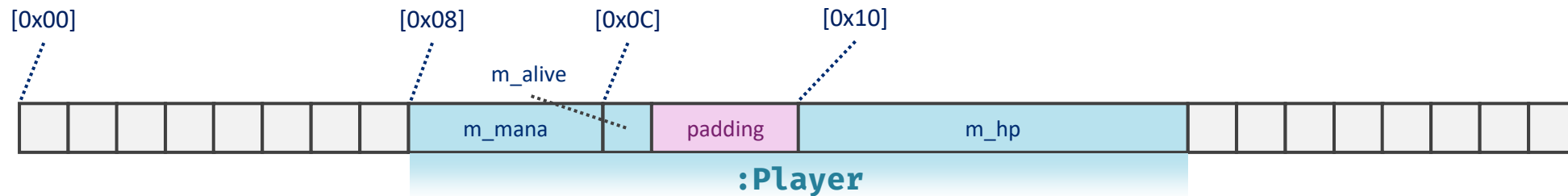
1 bajt

Pamäťová reprezentácia inštancie

- Zarovnanie typu získame operátorom `alignof`
- Aká je veľkosť a zarovnanie typu `Player`?

```
int main() {  
    std::cout << alignof(Player) << "\n"; // 8  
    std::cout << sizeof(Player) << "\n"; // 16  
}
```

```
class Player {  
public:  
    // ...  
  
private:  
    int m_mana;  
    bool m_alive;  
    double m_hp;  
};
```



1 bajt

Pamäťová reprezentácia inštancie

- Aký vplyv bude mať zmena usporiadania členských premenných?

```
int main() {  
    std::cout << alignof(Player) << "\n"; // ?  
    std::cout << sizeof(Player) << "\n";  // ?  
}
```

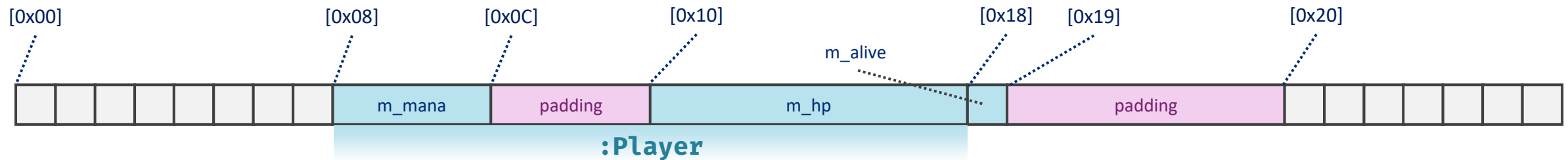
```
class Player {  
public:  
    // ...  
  
private:  
    int m_mana;  
    double m_hp;  
    bool m_alive;  
};
```

Pamäťová reprezentácia inštancie

- Poradie premenných ovplyvňuje veľkosť typu

```
int main() {  
    std::cout << alignof(Player) << "\n"; // 8  
    std::cout << sizeof(Player) << "\n"; // 24  
}
```

```
class Player {  
public:  
    // ...  
  
private:  
    int m_mana;  
    double m_hp;  
    bool m_alive;  
};
```



Zarovnanie a veľkosť

- Poradie deklarácie členských premenných ovplyvňuje veľkosť pamäťovej reprezentácie inštancií
 - Vhodnú veľkosť zvyčajne získame usporiadaním členských premenných zostupne alebo vzostupne podľa ich veľkosti ([sizeof](#))
- Zarovnanie triedy je dané maximom zo zarovnaní členských premenných
 - Pri bežnom programovaní sa ním nemusíme špeciálne zaoberať
 - Význam má pri manuálnom alokovaní pamäte – napríklad pri použití [object pool](#)
 - Dôležité je tiež pri práci so [SIMD inštrukciami](#)

Pamäťová reprezentácia členských funkcií

- Členské funkcie sú uložené v **kódovom segmente** pamäťového priestoru procesu
- Sú zdieľané pre všetky inštancie
- Viac sa im budeme venovať na nasledujúcej prednáške

```
Math::sum(std::vector<int> const&):  
    mov     rsi, QWORD PTR [rdi]  
    mov     rcx, QWORD PTR [rdi+8]  
    sub     rcx, rsi  
    je      .L7  
    sar     rcx, 2  
    lea     rax, [rcx-1]  
    cmp     rax, 2  
    jbe     .L8  
    mov     rdx, rcx  
    mov     rax, rsi  
    pxor    xmm0, xmm0  
    shr     rdx, 2  
    sal     rdx, 4  
    add     rdx, rsi
```

Vznik a zánik inštancií



Konštruktor

- Špeciálna metóda, ktorej úlohou je inicializovať členské premenné pri vzniku inštancie
- Jeho názov je totožný s názvom triedy a nemá návratový typ
- Na inicializáciu členských premenných môžeme použiť syntax [inicializácie členov](#)

```
Player::Player(double hp) {  
    m_mana = 100;  
    m_alive = true;  
    m_hp = hp;  
}
```

```
Player::Player(double hp) :  
    m_mana(100),  
    m_alive(true),  
    m_hp(hp)  
{  
}
```

Konštruktor

- Počet konštruktorov nie je obmedzený – musia sa líšiť typom alebo počtom parametrov
- Syntaxou inicializácie členov môžeme inicializáciu **delegovať** na iný konštruktor

```
Player::Player() :  
    Player(100.0)  
{  
}
```

```
class Player {  
public:  
    Player();  
    Player(double hp);  
  
private:  
    int m_mana;  
    bool m_alive;  
    double m_hp;  
};
```

```
Player::Player(double hp) :  
    m_mana(100),  
    m_alive(true),  
    m_hp(hp)  
{  
}
```


Konverzný konštruktor

- Niektoré konšuktory majú špeciálne prívlastky
- Jedným z nich je konverzný konštruktor – konštruktor, ktorý má práve jeden parameter
- Existencia takého konšuktora umožňuje implicitnú konverziu z typu parametra na typ triedy
- Takéto správanie takmer nikdy nechceme, preto konštruktor s práve jedným parametrom označujeme kľúčovým slovom **explicit**

Konverzný konštruktor

- V príklade sme typom `Degrees` chceli docieľať, aby funkcia `calcSin` mohla byť zavolaná iba so stupňami – vyjadrenie logiky v programe použitím silných typov
- Implicitná konverzia však umožní volanie funkcie aj s typom `double`

```
class Degrees {  
public:  
    Degrees(double value);  
    // ..  
private:  
    double m_value;  
};
```

```
Degrees::Degrees(double value) :  
    m_value(value)  
{  
}
```

```
double calcSin(Degrees d);
```

```
int main() {  
    double a = 45.5;  
    calcSin(a); // toto sme nechceli  
}
```

Konverzný konštruktor

- V príklade sme typom `Degrees` chceli docieľiť, aby funkcia `calcSin` mohla byť zavolaná iba so stupňami – vyjadrenie logiky v programe použitím silných typov
- Kľúčové slovo `explicit` zamedzí implicitnej konverzii

```
int main() {  
    Degrees a(45.5);  
    calcSin(a);  
}
```

```
class Degrees {  
public:  
    explicit Degrees(double value);  
    // ..  
private:  
    double m_value;  
};
```

```
Degrees::Degrees(double value) :  
    m_value(value)  
{  
}
```

```
double calcSin(Degrees d);
```

```
int main() {  
    double a = 45.5;  
    calcSin(a);  
}
```

Deštruktor

- Špeciálna metóda, ktorej úlohou je uvoľniť zdroje alokované v konštruktoe – zdroje, za ktoré je trieda zodpovedná
- Zdrojom je typicky alokovaná pamäť (inštancie tried) alebo otvorený súbor
- Jeho názov je totožný s názvom triedy s prefixom ~
- Nemá návratový typ ani parametre, *takmer vždy* je najviac jeden

```
class Display {  
public:  
    Display();  
    ~Display();  
private:  
    DisplayRow *m_rows[N_ROWS];  
};
```

```
Display::Display() {  
    for (int i = 0; i < N_ROWS; ++i) {  
        m_rows[i] = new DisplayRow();  
    }  
}
```

```
Display::~~Display() {  
    for (int i = 0; i < N_ROWS; ++i) {  
        delete m_rows[i];  
    }  
}
```

Generované konštruktory a deštruktory

- Ak trieda nepotrebuje deštruktor, nemusíme ho definovať
- Kompilátor za nás vygeneruje prázdny deštruktor (je lepšie nechať to naňho)
- Ak nedefinujeme žiadny konštruktor, kompilátor za nás vygeneruje prázdny bezparametrický konštruktor
- Z pohľadu čitateľnosti je lepšie členské premenné explicitne inicializovať – definovaním konštruktora alebo v mieste deklarácie – ako spoliehať sa na inicializáciu vygenerovaným konštruktorom

Vznik objektu (inštalácie)

- Vznik objektu (inštalácie) prebieha v dvoch* samostatných krokoch:
 1. Alokovanie pamäte – pre inštaláciu typu `T` je to `sizeof(T)` bajtov
 2. Inicializácia pamäte konštruktorom
- Objekt existuje až po 2. kroku
- Pri bežnom programovaní prebehnú oba kroky naraz a programátorovi sa javia ako jeden krok – volanie konšuktora je automatické

Výnimkou sú typy s implicitným životným cyklom *

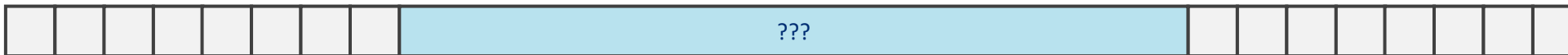
Vznik objektu (inštancie)

```
class Player {  
public:  
    Player(double hp);  
private:  
    int m_mana;  
    bool m_alive;  
    double m_hp;  
};
```

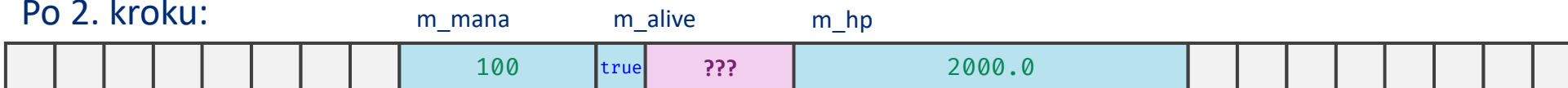
```
Player::Player(double hp) :  
    m_mana(100),  
    m_alive(true),  
    m_hp(hp)  
{}
```

```
Player *p = new Player(2000.0);
```

Po 1. kroku:



Po 2. kroku:



Zánik objektu (inštancie)

- Zánik objektu (inštancie) prebieha v dvoch krokoch
 1. Zrušenie objektu (inštancie) zavolaním jeho deštruktora
 2. Dealokovanie pamäte
- Pri bežnom programovaní prebehnú oba kroky naraz a programátorovi sa javia ako jeden krok – volanie konštruktora je automatické

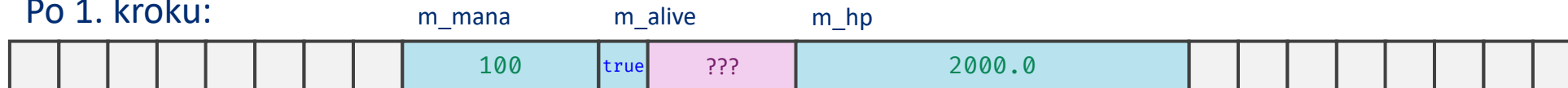
Zánik objektu (inštancie)

```
class Player {  
public:  
    Player(double hp);  
private:  
    int m_mana;  
    bool m_alive;  
    double m_hp;  
};
```

```
Player::Player(double hp) :  
    m_mana(100),  
    m_alive(true),  
    m_hp(hp)  
{}
```

```
Player *p = new Player(2000.0);  
delete p;
```

Po 1. kroku:



Po 2. kroku:



Zánik objektu (inštancie)

- Po zániku objektu zostávajú hodnoty premenných v pamäti
- Prístup k nim už však nie je možný – hoci nám to jazyk umožní, ide o nedefinované správanie

```
class Player {  
public:  
    explicit Player(double hp);  
    double getHP() const;  
private:  
    double m_hp;  
};
```

```
double Player::getHP() const {  
    return m_hp;  
}
```

```
int main() {  
    Player *p = new Player(2000.0);  
    delete p;  
    std::cout << p->getHP() << "\n"; // ??  
}
```

Hodnotová sémantika



Príklad

```
class Player {  
public:  
    explicit Player(int id);  
private:  
    int m_id;  
};
```

```
Player::Player(int id) :  
    m_id(id)  
{  
}
```

```
int main() {  
    Game *g = new Game(10);  
}
```

```
class Game {  
public:  
    explicit Game(int duration);  
    ~Game();  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

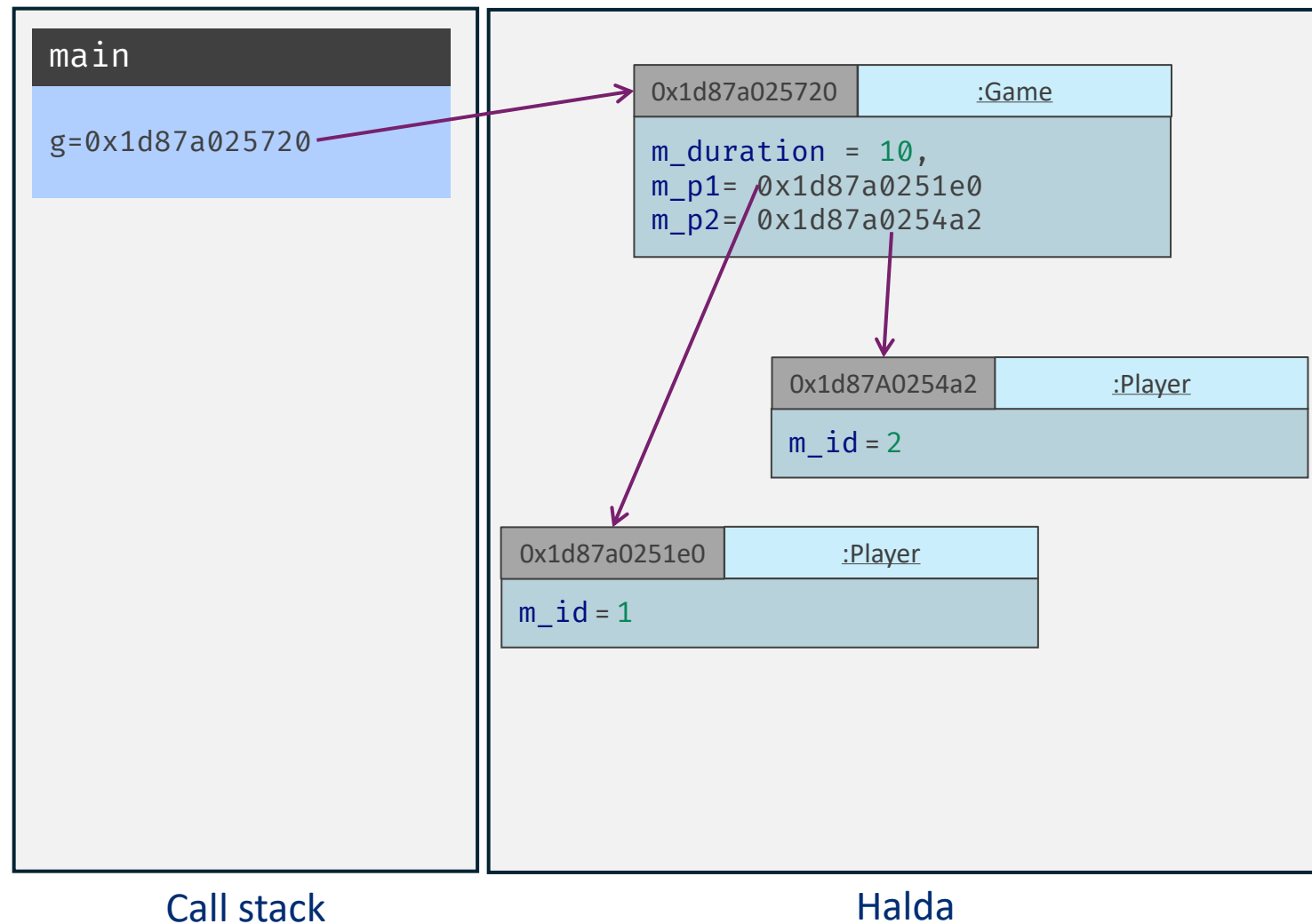
```
Game::Game(int duration) :  
    m_duration(duration),  
    m_p1(new Player(1)),  
    m_p2(new Player(2))  
{  
}
```

```
Game::~~Game() {  
    delete m_p1;  
    delete m_p2;  
}
```

Príklad

```
Game::Game(int duration) :  
    m_duration(duration),  
    m_p1(new Player(1)),  
    m_p2(new Player(2))  
{  
}
```

```
int main() {  
    Game *g = new Game(10);  
}
```



Hodnotová sémantika

- Opis vzniku inštancie nehovorí nič o tom, že pamäť musí byť alokovaná na halde
- Vytvorenie inštancie s použitím operátora **new** je v jazyku C++ len jedna z možností
- Druhou možnosťou je zadefinovanie premennej typu triedy
- Definícia premennej alokuje pamäť, v ktorej bude uložená jej hodnota
- Konštruktor sa zavolá automaticky rovnako ako pri použití **new**
- Deštruktor sa zavolá automaticky pri konci rozsahu platnosti premennej

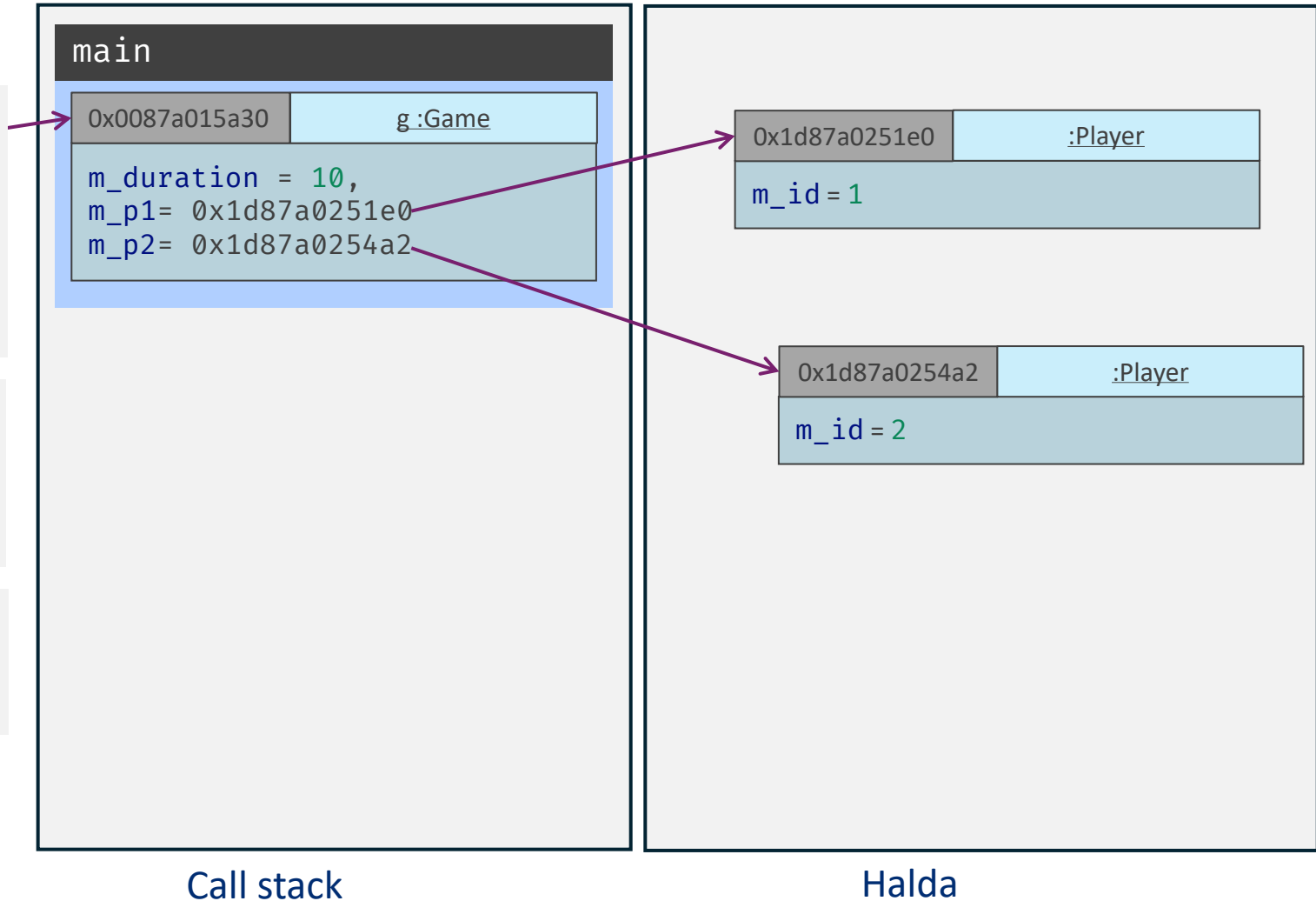
Príklad

```
Game::Game(int duration) :  
    m_duration(duration),  
    m_p1(new Player(1)),  
    m_p2(new Player(2))  
{  
}
```

```
Game::~~Game() {  
    delete m_p1;  
    delete m_p2;  
}
```

```
int main() {  
    Game g(10);  
}
```

Automatické vykonanie deštruktora,
pamäť je korektne uvoľnená.

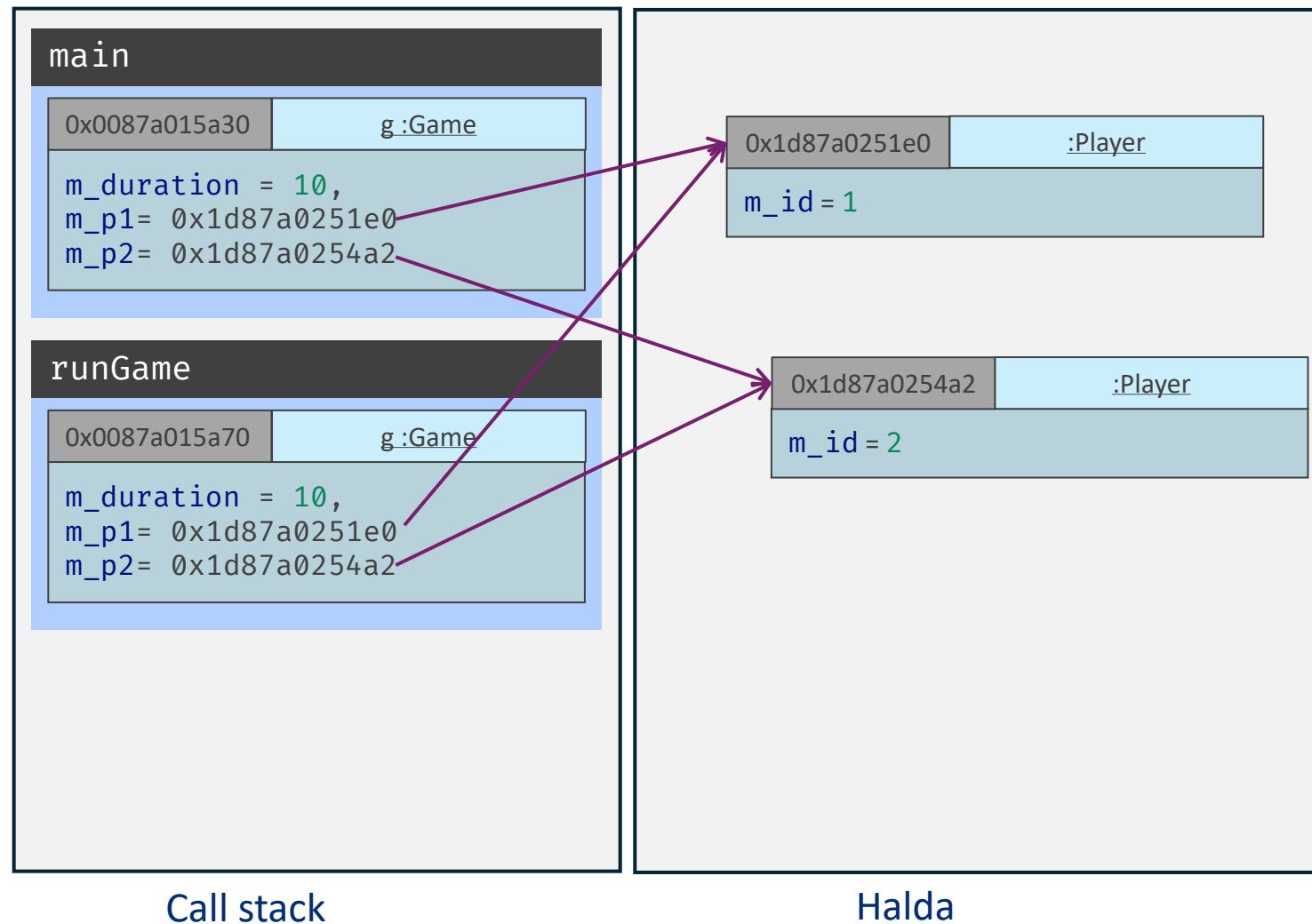


Príklad

```
void runGame(Game g) {  
    // ...  
}
```

```
int main() {  
    Game g(10);  
    runGame(g);  
}
```

Čo sa stane na konci volania funkcie
runGame?



Príklad

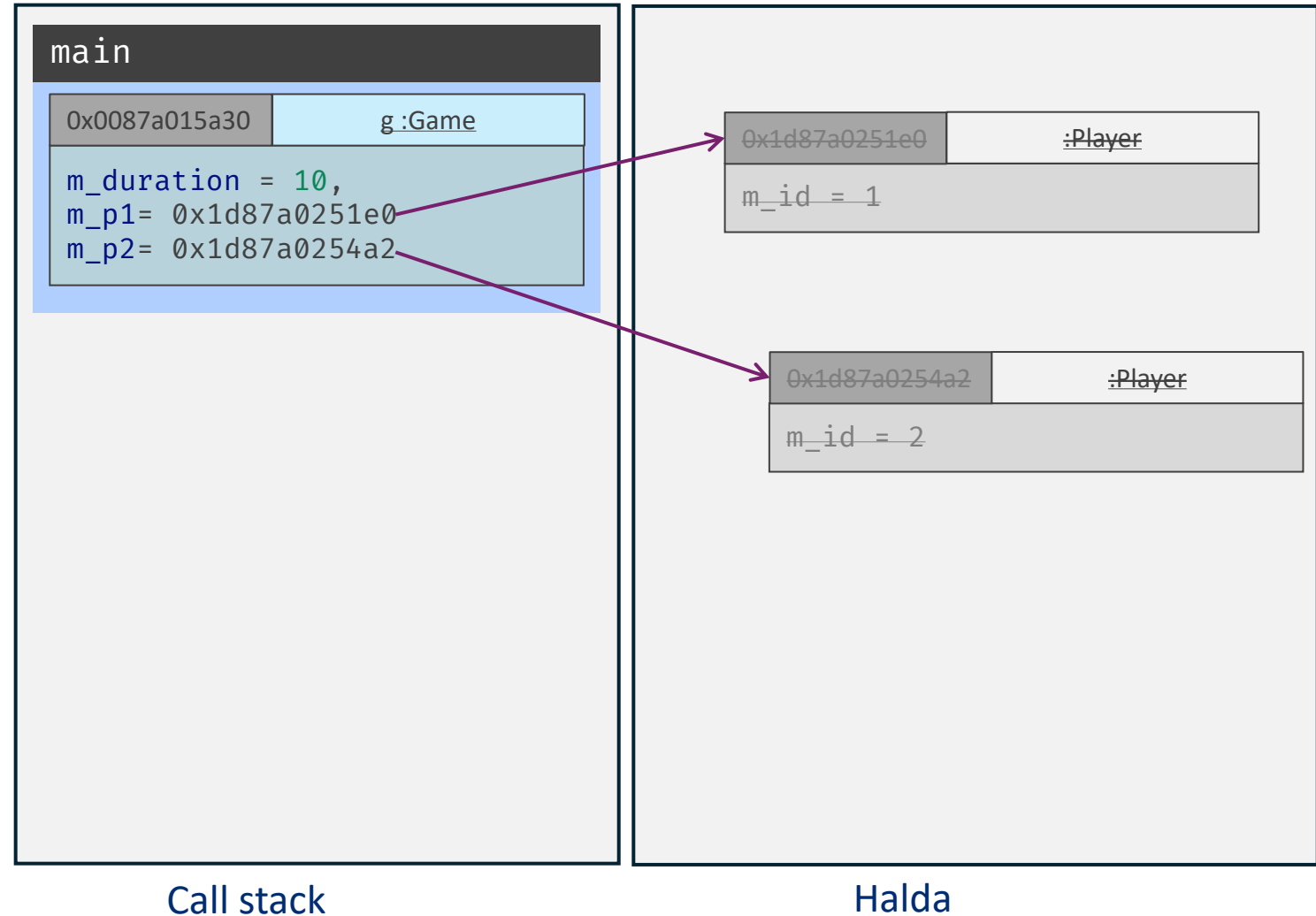
```
void runGame(Game g) {  
    // ...  
}
```

```
int main() {  
    Game g(10);  
    runGame(g);  
}
```



Deštruktor parametra `g` korektne vymazal inštancie triedy `Player`.

Čo sa stane na konci volania funkcie `main`?



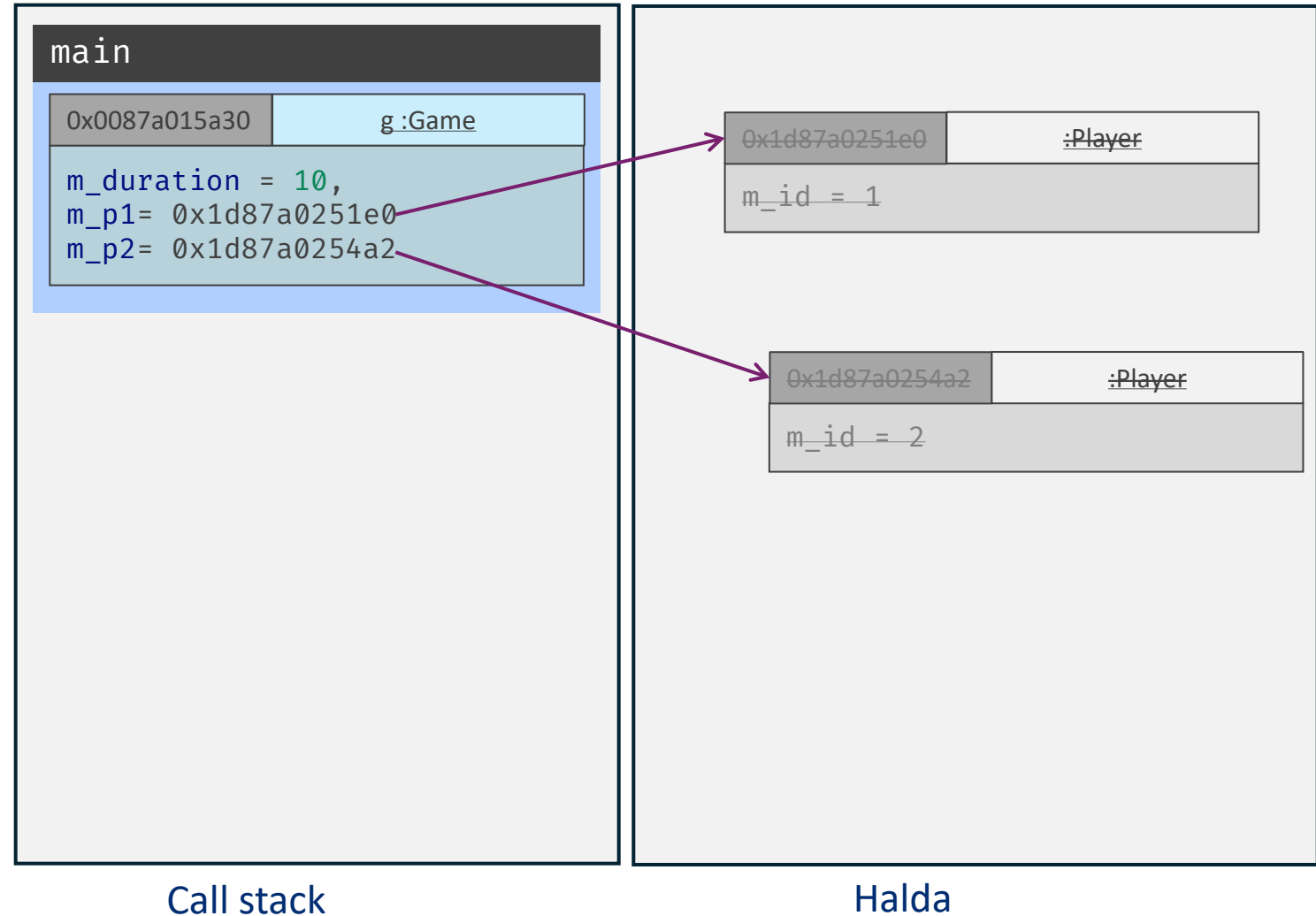
Príklad

```
void runGame(Game g) {  
    // ...  
}
```

```
int main() {  
    Game g(10);  
    runGame(g);  
}
```



V deštruktore sa pokúsime vymazať neexistujúce inštancie – dealokovať pamäť, ktorá už raz bola alokovaná – čo pravdepodobne vyústi v **behovú chybu**.



Príklad

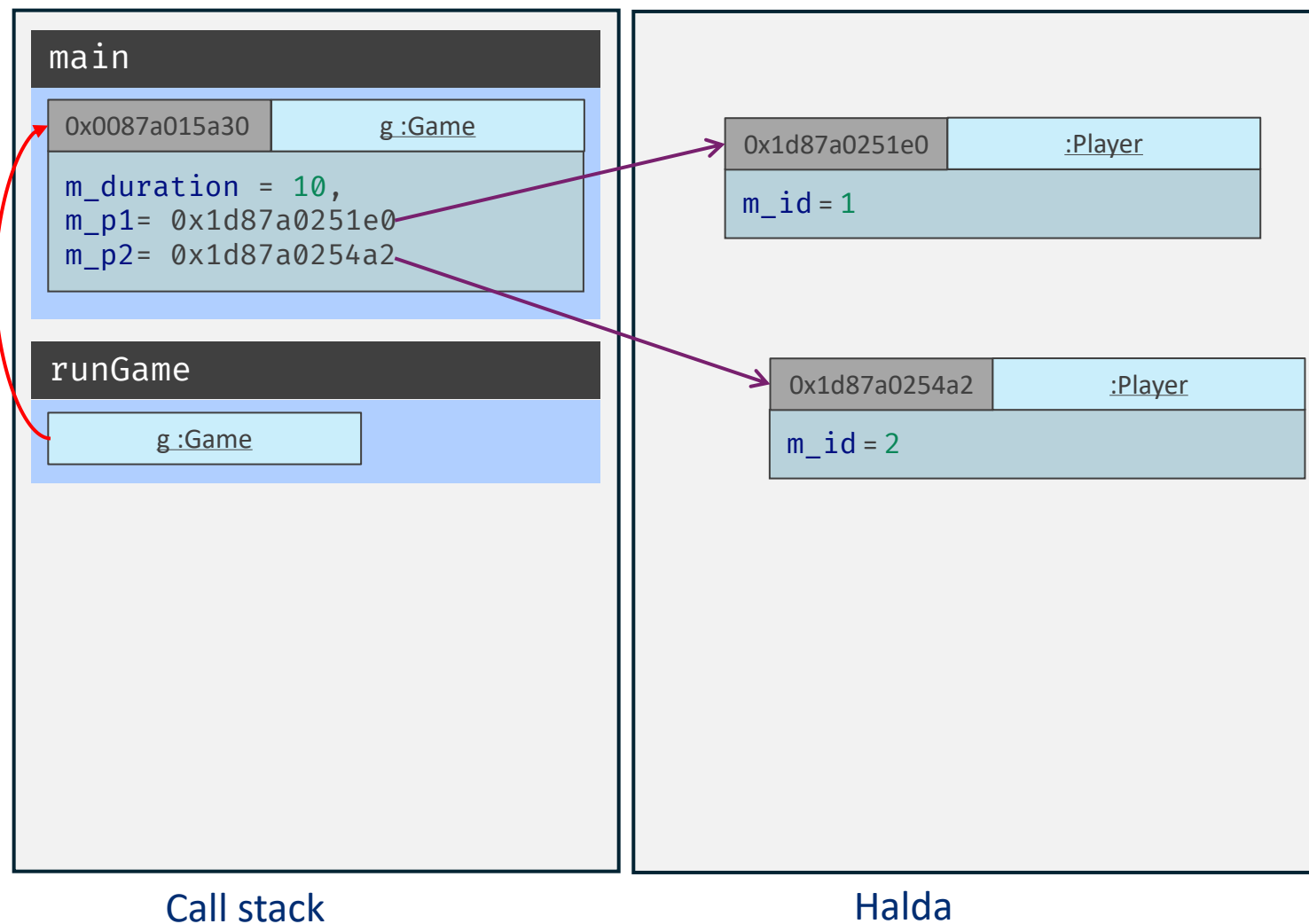


```
void runGame(Game &g) {  
    // ...  
}
```



```
int main() {  
    Game g(10);  
    runGame(g);  
}
```

Korektné riešenie je v tomto prípade použitie odkazu.



Kopírovací konštruktor

- Chyba v príklade vznikla, pretože pri odovzdávaní parametra hodnotou sa vytvorila **plytká kópia** inštancie triedy Game
- Plytká kópia zodpovedá **bajtovej kópii** reprezentácie inštancie
- Keďže sú však niektoré členské premenné iba ukazovatele, inštancie triedy **Player** boli zdieľané – mali dvoch vlastníkov, čo je logická chyba
- Na vytvorenie plytkej kópie sa použil automaticky vygenerovaný kopírovací konštruktor, ktorý – v tomto prípade – robí plytkú kópiu
- Riešením je manuálne definovanie kopírovacieho konštruktora, ktorý spraví **hlbokú kópiu**



Príklad

- Kopírovací konštruktor triedy **T** má presne danú syntax
`T(const T &other)`
- Volá sa automaticky, vždy keď vytvárame kópiu inštancie
- Ak trieda alokuje zdroje v konštruktoze, vždy je potrebné definovať ho manuálne a vytvoriť **hlbokú kópiu**

```
class Game {  
public:  
    explicit Game(int duration);  
    Game(const Game &other);  
    ~Game();  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

```
Game::Game(const Game &other) :  
    m_duration(other.duration),  
    m_p1(new Player(*other.m_p1)),  
    m_p2(new Player(*other.m_p2))  
{  
}
```

Potrebuje trieda **Player** tiež manuálne definovať kopírovací konštruktor?

Príklad

- V našom príklade by jeho definovanie vyriešilo problém s dvojitým uvoľnením hráčov
- Potenciálne by však vytvorilo problém s duplikovaním stavu hry a následnou nekonzistenciou
- Preto inštancie vždy posielame pomocou odkazu (ukazovateľa), ak je to možné, s `const` kvalifikátorom
- Kópie vytvárame, **len ak je to potrebné**
- Pri triede `Player` je automaticky vygenerovaný kopírovací konštruktor postačujúci

Práca s hodnotami

- (2) zavolá sa bezparametrický konštruktor
- (3) nezavolá sa bezparametrický konštruktor!
 - Išlo by o deklaráciu funkcie
- (6) pri práci s hodnotou k členom pristupujeme cez operátor .
- (7) pri práci s ukazovateľom k členom pristupujeme cez operátor ->
- (7) je skrátенý zápis (10)

```
class Game {  
public:  
    Game();  
    explicit Game(int duration);  
    void run();  
private:  
    int m_duration;  
    Player *m_p1;  
    Player *m_p2;  
};
```

```
1 int main() {  
2     Game g1;  
3     Game g1();  
4     Game g2(10);  
5     Game *g3 = new Game(10);  
6     g2.run();  
7     g3->run();  
8     Game &g4 = *g3;  
9     g4.run();  
10    (*g3).run();  
11 }
```

Príklad – vylepšenie

- Hodnoty môžeme použiť aj pri deklarácii členských premenných
- V tomto prípade budú inštalácie triedy `Player` súčasťou inštalácie triedy `Game`
- Deštruktory členských premenných *typu trieda* sú volané automaticky v deštruktore vlastníka (`~Game`) (v tomto prípade vygenerovanom)

```
class Game {  
public:  
    explicit Game(int duration);  
private:  
    int m_duration;  
    Player m_p1;  
    Player m_p2;  
};
```

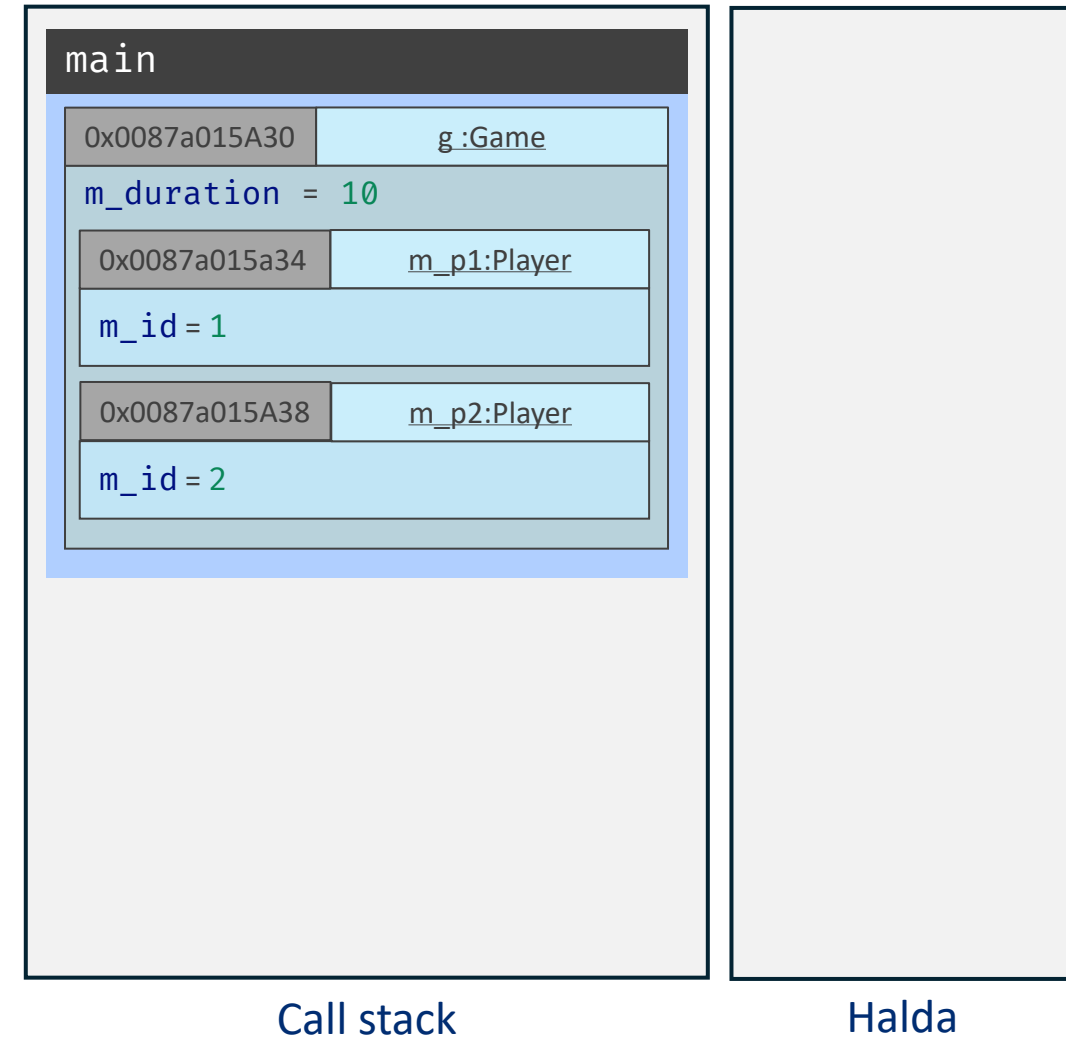
```
Game::Game(int duration) :  
    m_duration(duration),  
    m_p1(1),  
    m_p2(2)  
{  
}
```

Pomocou syntaxe inicializácie členov odovzdáme hodnoty parametrov ich konštruktorov. V tele konštruktora to nie je možné!

Príklad

```
class Game {  
public:  
    explicit Game(int duration);  
private:  
    int m_duration;  
    Player m_p1;  
    Player m_p2;  
};
```

```
int main() {  
    Game g(10);  
}
```



Hodnotová sémantika

- Použitie hodnotovej sémantiky môže zjednodušiť kód
- Vhodné použitie môže výrazne vylepšiť **pamäťovú lokalitu**
- Je potrebné zabrániť **zbytočnému kopírovaniu**, ktoré môže vytvoriť logické chyby
- Tiež je potrebné myslieť na **veľkosť inštancií** pri ich umiestňovaní na zásobník
- Je vhodná na vyjadrenie **kompozície**, ak má byť však inštancia dostupná na viacerých miestach programu, je na týchto miestach potrebné použiť **nepriamy prístup** – odkaz alebo ukazovateľ

Zhrnutie

- Pamäťová reprezentácia inštancie
 - Najviac sa nás dotýka veľkosť ovplyvnená usporiadaním
- Deštruktor
 - Ak v konštruktore alokujeme zdroje, je potrebné ich uvoľniť v deštruktore
- Hodnotová sémantika
 - Nám pomáha zjednodušiť kód a automatizovať správu pamäte
 - Má však úskalia, ktoré je potrebné zohľadniť

Ďakujem za pozornosť

Priestor na vaše otázky

